

UNIVERSITÀ DI BOLOGNA



School of Engineering
Master Degree in Automation Engineering

Optimal Control and Reinforcement Learning
Optimal Control of a Flexible Robotic Arm

Professor: **Giuseppe Notarstefano**
Tutor: **Marco Falotico**

Students:
Sharon Patrone
Martina Raffaele
Sarah Mercolino

Academic year 2025/2026

Abstract

This report presents the modelling, trajectory generation, and trajectory tracking of a flexible robotic arm using optimal control techniques. The system considered is a planar two-link robotic arm actuated by a torque applied at the first joint only, with the second joint being passive. The non-linear continuous-time dynamics, described by Euler–Lagrange equations, are discretized via a Forward Euler scheme. Jacobians of the discrete-time dynamics are computed using CasADi automatic differentiation.

Two trajectory generation tasks are addressed. In the first, an optimal transition between two equilibrium configurations is computed starting from a sigmoid-interpolated reference curve. In the second, a quasi-static trajectory is constructed as a sequence of equilibria and used both as a reference and as the basis for an LQR-based initial guess, before applying the same Newton-like optimization. In both tasks, the Newton descent direction is obtained by solving an affine Linear Quadratic Regulator subproblem via backward Riccati recursion, the direction is applied to the nonlinear dynamics through a closed-loop rollout, and the step size is selected using an Armijo line search.

The optimal trajectory produced in the second task is then tracked under initial perturbations using both a Linear Time-Varying LQR and a Model Predictive Controller. The MPC is formulated in error coordinates over a finite prediction horizon and explicitly enforces bounds on the actual control input. Both controllers are tested under the same five initial perturbations, enabling a comparison of their tracking behaviour while accounting for the MPC input constraints. An animation of the LQR-controlled motion is also implemented to provide a qualitative validation of the results.

Contents

Introduction	5
1 Task 0 – Problem Setup	6
1.1 Dynamic model	6
1.2 Forward Euler discretization	6
1.3 Linearization via CasADi	7
2 Task 1 – Trajectory Generation I	8
2.1 Equilibrium computation	8
2.2 Desired reference curve	8
2.3 Optimal control formulation	9
2.4 Newton-like algorithm	9
2.5 Armijo line search	10
2.6 Initial guess	10
2.7 Numerical results	10
3 Task 2 – Trajectory Generation II	14
3.1 Quasi-static reference trajectory	14
3.2 LQR-based initial guess	14
3.3 Newton optimization	15
3.4 Numerical results	15
4 Task 3 – Trajectory Tracking via LQR	19
4.1 LTV-LQR tracking controller	19
4.2 Tracking results	19
5 Task 4 – Trajectory Tracking via MPC	22
5.1 MPC formulation	22
5.2 MPC results and comparison with LQR	23
6 Task 5 – Animation	25
Conclusions	26
Bibliography	27

Introduction

Flexible robotic manipulators are widely used in applications that require light structures and precise motion control. The dynamics is nonlinear and the motion of the two links is coupled through inertia, Coriolis, centrifugal, friction, and gravity terms. Moreover, the underactuated nature of the system, with only one control input acting on the first joint, makes trajectory planning and stabilization more demanding. The work is organized into six main tasks: Task 0 covers the discretization and implementation of the nonlinear dynamics; Tasks 1 and 2 address trajectory generation via a Newton-like optimal control algorithm; Tasks 3 and 4 focus on trajectory tracking via LQR and MPC; Task 5 provides an animation of the controlled motion.

Contributions

All members of the group contributed collaboratively to the development of the project and to the preparation of this report. The authors would like to thank their tutor, Dr. Marco Falotico, for his guidance and suggestions throughout the project.

Chapter 1

Task 0 – Problem Setup

1.1 Dynamic model

The flexible arm is described by the state vector

$$x = [\theta_1 \quad \theta_2 \quad \dot{\theta}_1 \quad \dot{\theta}_2]^\top,$$

where θ_1 is the angle of the first link with respect to the vertical direction, θ_2 is the relative angle of the second link with respect to the first one, and $\dot{\theta}_1$, $\dot{\theta}_2$ are the corresponding angular velocities. The control input is the torque applied to the first joint:

$$u = \tau.$$

The continuous-time dynamics follows from the Euler–Lagrange equations

$$M(\theta_1, \theta_2) \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} + C(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2) + F \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} + G(\theta_1, \theta_2) = \begin{bmatrix} u \\ 0 \end{bmatrix}.$$

The matrix M relates applied torques to angular accelerations. The vector C contains Coriolis and centrifugal nonlinear coupling terms. The matrix F models viscous friction proportional to angular velocity, while G collects the gravity torques acting on each link. The zero in the second component of the input vector reflects the fact that the second joint is passive and cannot be directly actuated.

The model parameters used throughout this project correspond to Parameter Set 1 of the assignment: $m_1 = m_2 = 1 \text{ kg}$, $l_1 = l_2 = 1 \text{ m}$, $r_1 = r_2 = 0.5 \text{ m}$, $I_1 = I_2 = 0.33 \text{ kg m}^2$, $g = 9.81 \text{ m/s}^2$, $f_1 = f_2 = 0.1$.

1.2 Forward Euler discretization

The continuous-time dynamics were discretized using the Forward Euler method with sampling time

$$\Delta t = 10^{-2} \text{ s}.$$

The discrete-time model is

$$x_{t+1} = f(x_t, u_t),$$

where the position states are updated as

$$\theta_{1,t+1} = \theta_{1,t} + \Delta t \dot{\theta}_{1,t}, \quad \theta_{2,t+1} = \theta_{2,t} + \Delta t \dot{\theta}_{2,t},$$

and the angular velocities are updated using the accelerations obtained by solving the dynamic equation at each step:

$$\ddot{\theta} = M(\theta)^{-1} \left(\begin{bmatrix} u \\ 0 \end{bmatrix} - C(\theta, \dot{\theta}) - F\dot{\theta} - G(\theta) \right).$$

Forward Euler was selected for its simplicity and low computational cost.

1.3 Linearization via CasADi

The trajectory generation and tracking algorithms require the Jacobians of the discrete-time dynamics at each time step:

$$A_t = \frac{\partial f}{\partial x}(x_t, u_t), \quad B_t = \frac{\partial f}{\partial u}(x_t, u_t).$$

They were computed using CasADi algorithmic differentiation, rather than finite differences, as it provided better results.

Chapter 2

Task 1 – Trajectory Generation I

2.1 Equilibrium computation

The first trajectory generation task requires identifying two equilibrium configurations of the arm. At an equilibrium, all velocities and accelerations are zero:

$$\dot{\theta}_1 = \dot{\theta}_2 = \ddot{\theta}_1 = \ddot{\theta}_2 = 0.$$

The equilibrium condition therefore reduces to just gravity compensation. For a prescribed value of θ_1 , the passive joint angle θ_2 is determined by requiring that the gravity torque at the second joint vanishes:

$$G_2(\theta_1, \theta_2) = 0.$$

The equilibrium input at the first joint is then given by

$$u_{eq} = G_1(\theta_1, \theta_2).$$

This nonlinear equation in θ_2 was solved numerically using `scipy.optimize.fsolve`. Two symmetric configurations were selected, corresponding to $\theta_1 = +25^\circ$ and $\theta_1 = -25^\circ$.

2.2 Desired reference curve

A smooth reference curve was constructed by interpolating between the two equilibrium configurations. A sigmoid-based transition was adopted: the interpolation variable $s_t \in [0, 1]$ is held constant at zero for an initial portion of the horizon, follows a normalized sigmoid profile during the middle portion, and is held constant at one for the final portion. The desired angles are then obtained as

$$\theta_{i,t}^{ref} = (1 - s_t) \theta_i^{eq,1} + s_t \theta_i^{eq,2}, \quad i = 1, 2$$

and the desired angular velocities are computed by numerical differentiation of the angle profiles. The desired input is interpolated using the same sigmoid profile.

2.3 Optimal control formulation

The objective is to find a dynamically feasible trajectory that follows the desired reference as closely as possible. The cost function is

$$J = \sum_{t=0}^{T-1} \ell(x_t, u_t) + \ell_T(x_T),$$

with stage cost

$$\ell(x_t, u_t) = \frac{1}{2}(x_t - x_t^{ref})^T Q (x_t - x_t^{ref}) + \frac{1}{2}(u_t - u_t^{ref})^T R (u_t - u_t^{ref})$$

and terminal cost

$$\ell_T(x_T) = \frac{1}{2}(x_T - x_T^{ref})^T Q_T (x_T - x_T^{ref}).$$

The weighting matrices were set to

$$Q = \text{diag}(1, 1, 0.7, 0.7), \quad R = \text{diag}(0.1).$$

The terminal cost matrix Q_T was obtained as the solution of a discrete algebraic Riccati equation (DARE) evaluated at the final equilibrium, so that the terminal penalty reflects the infinite-horizon cost of stabilizing the system from that point onward.

2.4 Newton-like algorithm

The nonlinear optimal control problem is solved using a Newton-like method in the shooting framework, where states are treated as implicit functions of the input sequence through the nonlinear dynamics. This reduces the decision variables to just the input trajectory $\{u_t\}$.

At each iteration, the dynamics is linearized along the current trajectory and the cost is approximated quadratically. The resulting local problem is an affine LQR subproblem, solved via backward Riccati recursion. This yields a feedback matrix K_t and a feedforward term σ_t at each time step, which together define the Newton descent direction in the input space. In accordance with the project guidelines, the local quadratic model uses only the Hessians of the cost function, while the nonlinear dynamics enter the subproblem through their first-order linearization.

The direction is applied to the nonlinear system through a closed-loop rollout:

$$u_t^{new} = u_t + K_t(x_t^{new} - x_t) + \gamma \sigma_t, \quad x_{t+1}^{new} = f(x_t^{new}, u_t^{new}).$$

This structure ensures that the updated trajectory remains dynamically feasible at every iteration, since it is generated by forward simulation of the true nonlinear dynamics. The scalar $\gamma \in (0, 1]$ is the step size selected by the Armijo line search.

2.5 Armijo line search

To prevent excessively large Newton steps that could violate the validity of the local linear approximation or increase the cost, an Armijo line search is applied at each iteration. A step size γ is accepted if the sufficient decrease condition

$$J(u + \gamma \Delta u) \leq J(u) + c \gamma \nabla J(u)^T \Delta u$$

is satisfied. The parameters used are $\gamma_0 = 1$, $\beta = 0.7$, $c = 0.5$, with a maximum of 20 backtracking steps. If the condition is not met, γ is multiplied by β and the check is repeated.

2.6 Initial guess

For Task 1, the initial feasible trajectory is obtained by propagating the nonlinear dynamics forward from the initial state x_0 using the reference input u^{ref} directly. Although this trajectory is not optimal, it is dynamically consistent and provides a reasonable starting point for the Newton iterations.

2.7 Numerical results

The Newton algorithm was run for a time horizon of $T_f = 8.5$ s with a convergence threshold of 10^{-4} on the norm of the descent direction. Convergence was reached in 10 iterations.

Figure 2.1 shows the Armijo descent plots at iterations 0, 1, 8 and 9. At iteration 0, the cost of the closed-loop step lies above the Armijo line for the full step size $\gamma = 1$. The accepted step size ($\gamma = 0.7$) satisfies the sufficient decrease condition. As the iterations proceed, the curvature of the cost curve diminishes, indicating that the trajectory is approaching a stationary point and the linear approximation of the cost becomes more accurate.

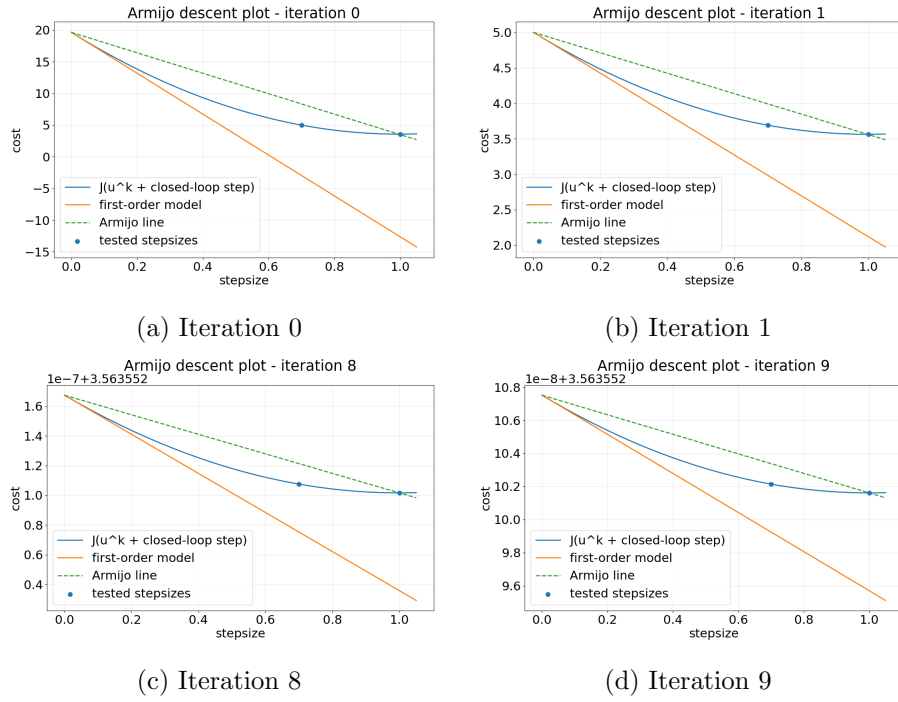


Figure 2.1: Task 1: Armijo descent plots at selected iterations.

Figure 2.2 shows the cost evolution along Newton iterations on a semi-logarithmic scale. The cost decreases sharply in the first two iterations and then stabilizes.

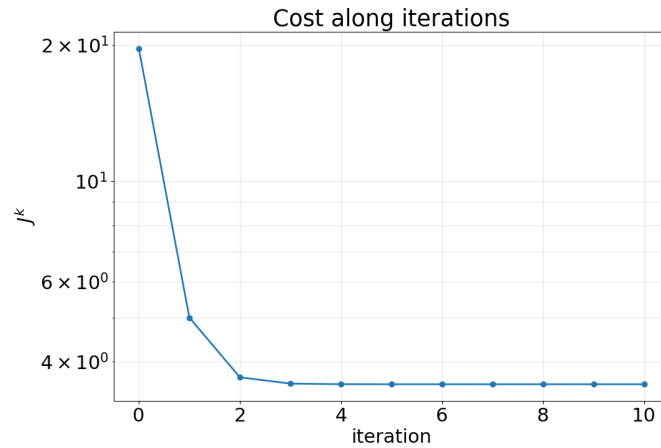


Figure 2.2: Task 1: cost along Newton iterations.

Figure 2.3 reports the norm of the descent direction and the norm of the reduced gradient as functions of the iteration index. Both quantities de-

crease monotonically on a logarithmic scale, confirming convergence toward a stationary point.

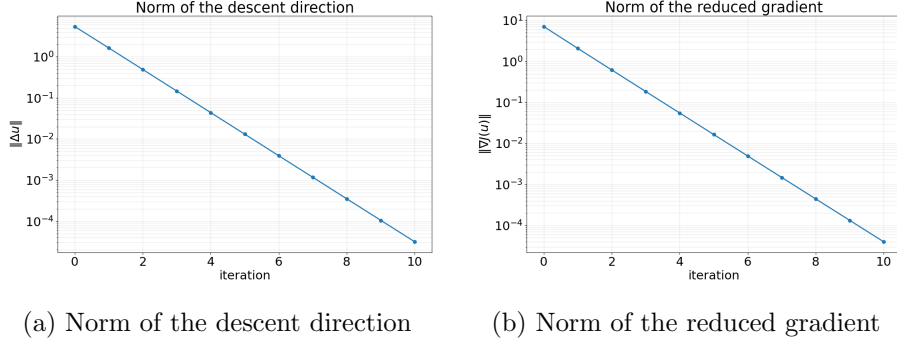


Figure 2.3: Task 1: convergence indicators along Newton iterations.

The optimal trajectory is compared with the desired reference in Figure 2.4. The optimal θ_1 and θ_2 profiles closely follow the sigmoid-interpolated reference, while the velocity and input profiles are modified to satisfy the nonlinear dynamics.

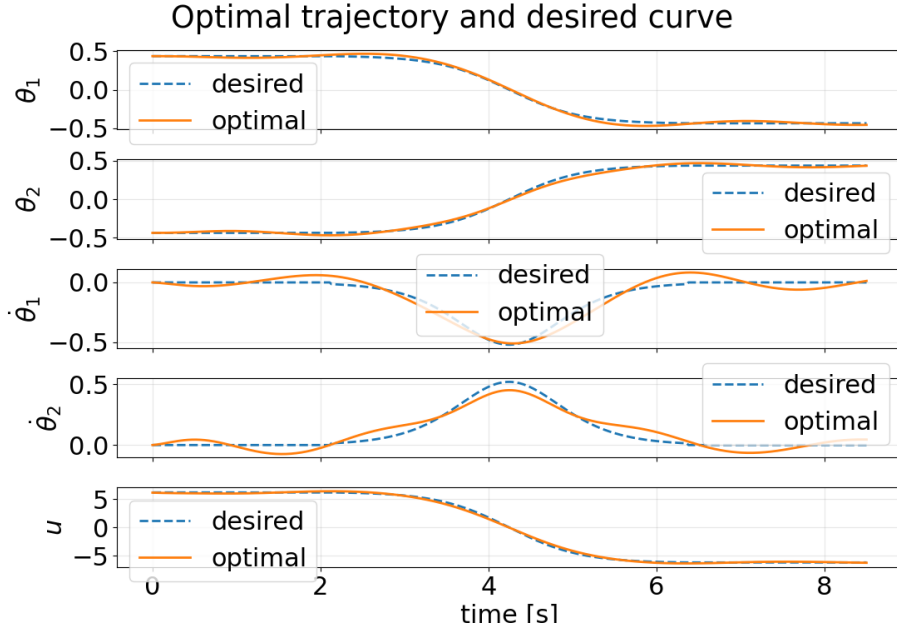


Figure 2.4: Task 1: optimal trajectory and desired reference curve.

Figure 2.5 illustrates the convergence of intermediate trajectories toward the optimal solution. At iteration 0, the trajectory (obtained from the reference input rollout) deviates noticeably from the desired curve. After just

a few iterations, the trajectories align closely with the optimal solution.

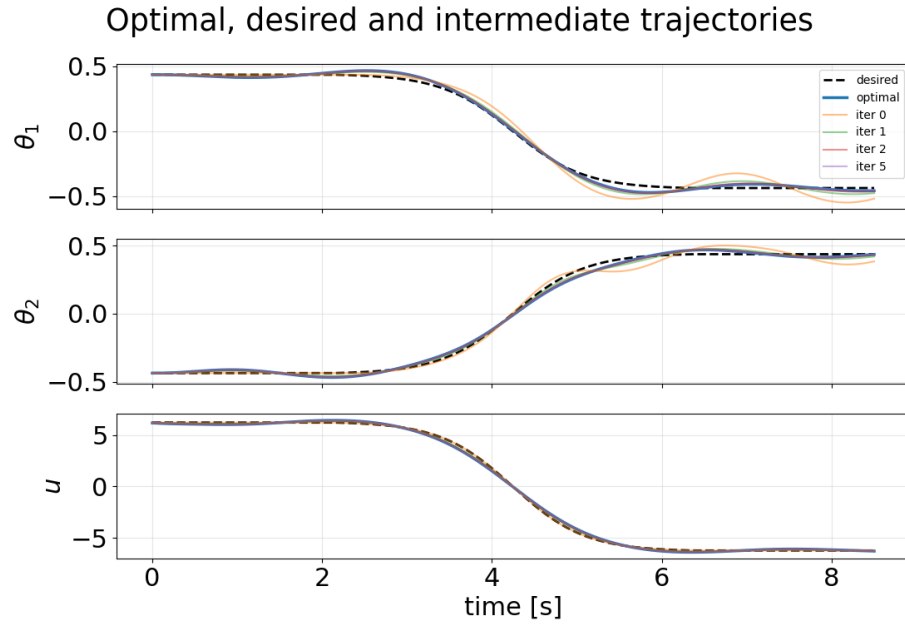


Figure 2.5: Task 1: desired curve, intermediate trajectories, and optimal solution.

Chapter 3

Task 2 – Trajectory Generation II

3.1 Quasi-static reference trajectory

In the second trajectory generation task, a more structured desired curve is constructed using a quasi-static approach: instead of directly interpolating between the two equilibrium points, intermediate equilibrium configurations are generated by solving the equilibrium condition at each time step while θ_1 follows a smooth sigmoid profile. This yields a sequence (x_t^{qs}, u_t^{qs}) that is physically meaningful at every instant, since each point corresponds to a configuration where the arm would be at rest (although the full sequence is not dynamically feasible).

The desired angular velocities in x^{ref} are obtained by numerical differentiation of θ_1^{qs} and θ_2^{qs} , replacing the null velocities of the quasi-static equilibria with a smooth estimate of the motion rate.

3.2 LQR-based initial guess

A key improvement with respect to Task 1 concerns the construction of the initial feasible trajectory. Since the quasi-static reference is more challenging to track, using it directly as an initial guess would result in a trajectory that is far from feasible and would slow down convergence.

To obtain a better starting point, the quasi-static trajectory is first tracked with a Linear Time-Varying LQR (LTV-LQR) controller designed along (x^{qs}, u^{qs}) . The resulting closed-loop simulation generates a feasible trajectory that remains close to the reference and provides an effective initial guess for the subsequent Newton optimization.

The cost matrices used for the LTV-LQR initial guess are

$$Q_{reg} = \text{diag}(1, 1, 0.7, 0.7), \quad R_{reg} = \text{diag}(0.1),$$

with terminal weight Q_T^{reg} computed via DARE at the final quasi-static point.

3.3 Newton optimization

The same Newton-like algorithm described in Task 1 is applied with identical cost matrices Q , R , and terminal weight Q_T computed via DARE at the final equilibrium of the quasi-static trajectory. The algorithm iterates until the norm of the descent direction falls below the threshold 10^{-4} .

3.4 Numerical results

The Newton algorithm converged in 11 iterations. Figures 3.1 and 3.2 show the Armijo descent plots for the first two and the last two iterations, respectively. At iteration 0, the initial cost is significantly higher than in Task 1, reflecting the fact that the quasi-static LQR initial guess, while dynamically feasible, is still far from optimal.

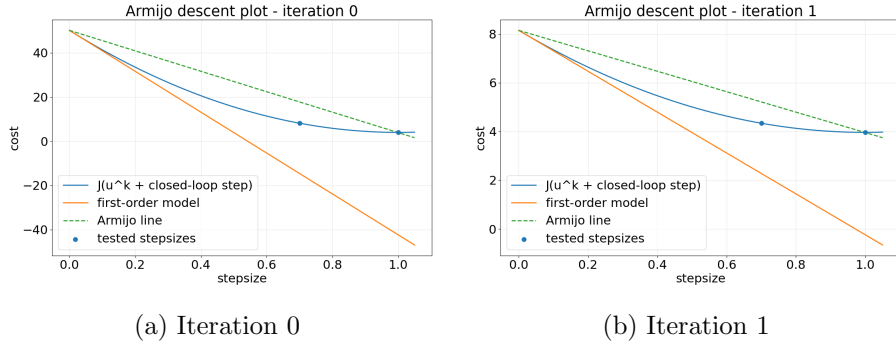


Figure 3.1: Task 2: Armijo descent plots at early iterations.

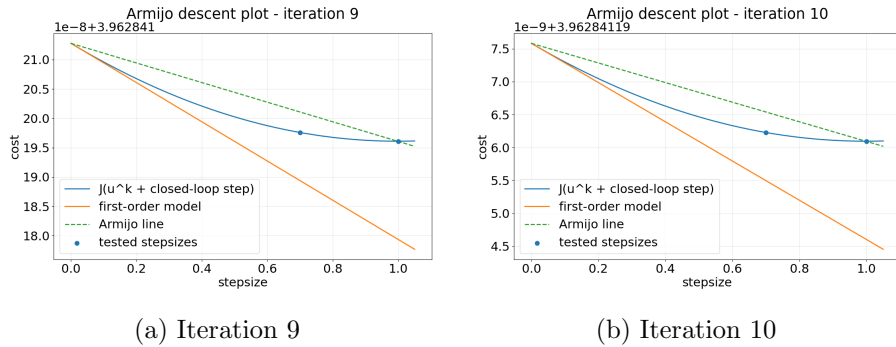


Figure 3.2: Task 2: Armijo descent plots at late iterations.

Figure 3.3 shows the cost evolution. The cost drops sharply in the first two iterations, from approximately 50 to below 4, and then decays more gradually, stabilizing after iteration 3.

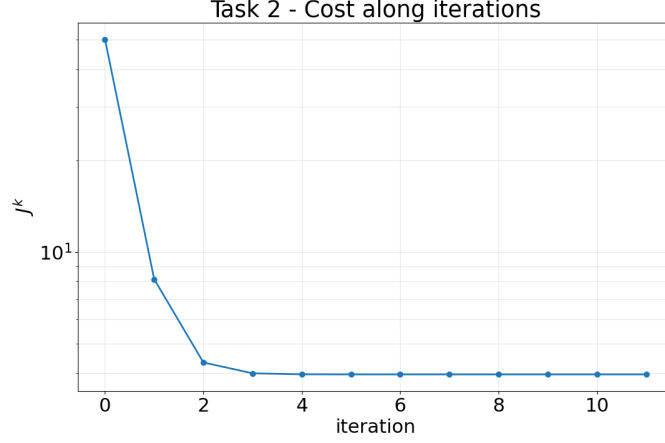


Figure 3.3: Task 2: cost along Newton iterations.

Figure 3.4 shows the norm of the descent direction and the norm of the reduced gradient. Both converge monotonically to values below 10^{-4} , confirming that the algorithm reaches a stationary point. The decay rate is slightly slower than in Task 1, consistent with the increased complexity of the quasi-static reference.

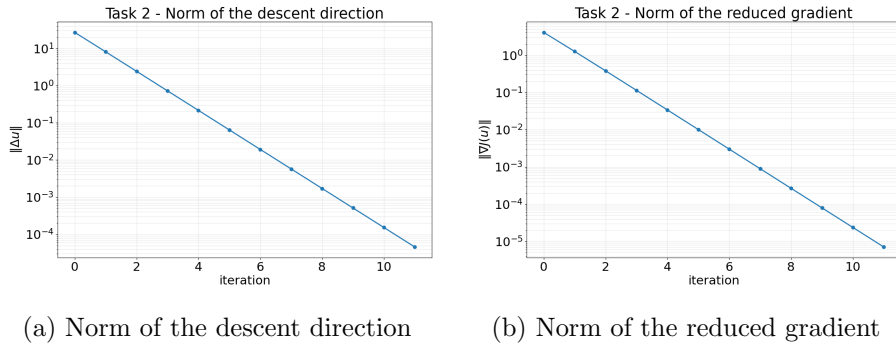


Figure 3.4: Task 2: convergence indicators along Newton iterations.

The optimal trajectory versus the desired quasi-static curve is shown in Figure 3.5. The angle profiles θ_1 and θ_2 track the reference closely. The velocity and input profiles are modified more substantially, reflecting the dynamic corrections needed to execute the motion in finite time.

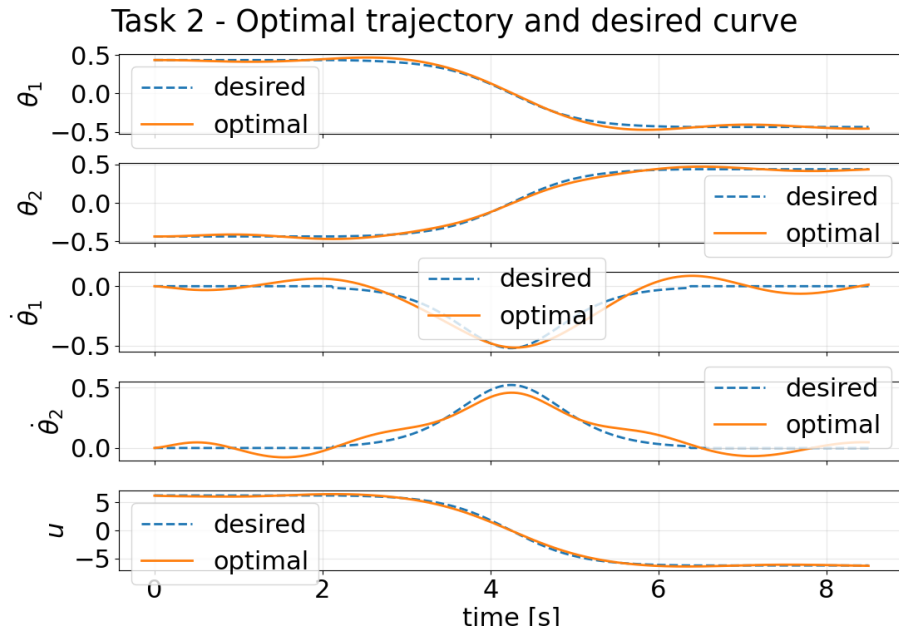


Figure 3.5: Task 2: optimal trajectory and desired quasi-static reference.

Figure 3.6 illustrates the intermediate trajectories. After just 1 iteration, the tracked trajectory follows the desired curve reasonably well, and from iteration 2 onward the corrections are imperceptible.

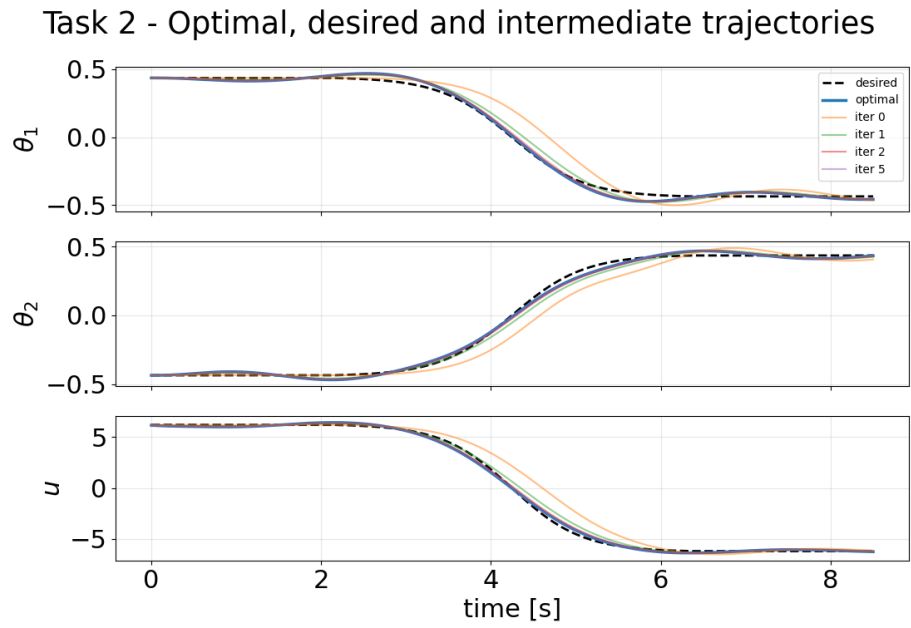


Figure 3.6: Task 2: desired curve, intermediate trajectories, and optimal solution.

Chapter 4

Task 3 – Trajectory Tracking via LQR

4.1 LTV-LQR tracking controller

The optimal trajectory (x^{gen}, u^{gen}) generated in Task 2 serves as the nominal reference for trajectory tracking. The nonlinear dynamics are linearized along this trajectory to obtain the time-varying matrices A_t^{gen} and B_t^{gen} . A Linear Time-Varying LQR problem is then solved to determine the optimal feedback gain sequence $\{K_t\}$ via backward Riccati recursion:

$$\Delta x_{t+1} = A_t^{gen} \Delta x_t + B_t^{gen} \Delta u_t.$$

The resulting control law applied to the nonlinear system is

$$u_t = u_t^{gen} + K_t (x_t - x_t^{gen}).$$

The cost matrices are

$$Q_{reg} = \text{diag}(1, 1, 0.7, 0.7), \quad R_{reg} = \text{diag}(0.1),$$

with terminal weight obtained from the DARE at the final point of the generated trajectory.

4.2 Tracking results

The controller was tested under five different initial perturbations applied to the nominal initial state x_0^{gen} : a 2° perturbation on θ_1 , a 2° perturbation on θ_2 , a combined $\pm 2^\circ$ perturbation on both angles, a 5° perturbation on θ_1 , and a 10° perturbation on θ_1 .

Figure 4.1 shows the tracked trajectories alongside the generated reference. For the smaller perturbations (2°), the controlled trajectories are

nearly indistinguishable from the nominal one, particularly in the angle plots. The larger perturbations (5° and 10° on θ_1) produce visible deviations in the velocity states $\dot{\theta}_1$ and $\dot{\theta}_2$ at early times, with the 10° case showing the most significant transient. In all cases, however, the angle trajectories converge toward the reference and the control input remains smooth.

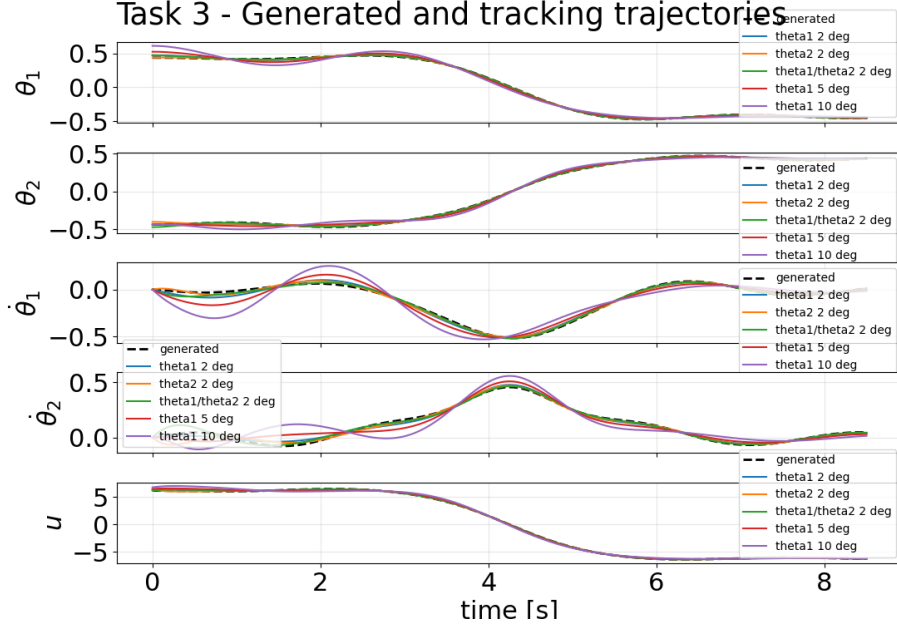


Figure 4.1: Task 3: generated trajectory and LQR tracking trajectories under five perturbations.

Figure 4.2 shows the tracking error norm $\|x_t - x_t^{gen}\|$ on a semi-logarithmic scale. All five curves exhibit a decreasing trend, confirming that the LTV-LQR controller successfully attenuates the initial perturbations. The combined 2° perturbation shows the fastest convergence rate, while the 10° perturbation on θ_1 produces the largest transient error and the slowest decay. The oscillatory envelope visible in the error curves is due to the flexible dynamics of the second link, which introduce a lightly damped mode that is excited by the initial offset.

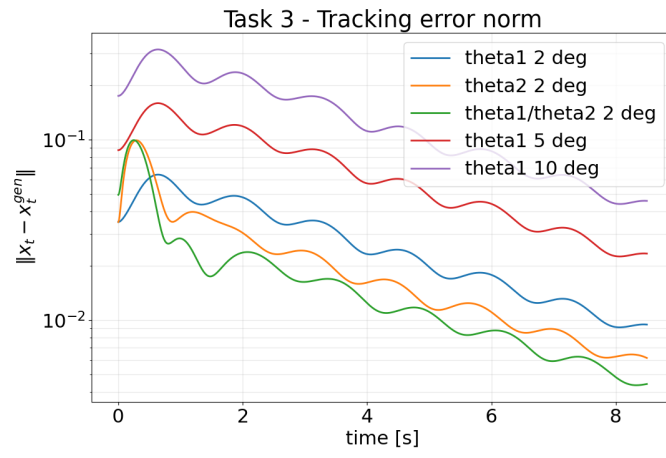


Figure 4.2: Task 3: LQR tracking error norm for five initial perturbations.

Chapter 5

Task 4 – Trajectory Tracking via MPC

5.1 MPC formulation

The same generated trajectory (x^{gen}, u^{gen}) was also tracked using a constrained receding-horizon Model Predictive Controller. The optimization is formulated in error coordinates,

$$\Delta x_t = x_t - x_t^{gen}, \quad \Delta u_t = u_t - u_t^{gen},$$

and uses the time-varying linearized dynamics obtained along the generated trajectory:

$$\Delta x_{t+1} = A_t^{gen} \Delta x_t + B_t^{gen} \Delta u_t.$$

At each time step, a finite-horizon quadratic optimization problem is solved with prediction horizon $N_p = 5$. The cost matrices are identical to those used in the LQR design:

$$Q_{MPC} = \text{diag}(1, 1, 0.7, 0.7), \quad R_{MPC} = \text{diag}(0.1),$$

with the same DARE-based terminal weight Q_T^{MPC} evaluated at the final point of the generated trajectory.

Unlike the LQR controller, the MPC explicitly constrains the actual control input

$$u_t = u_t^{gen} + \Delta u_t$$

according to

$$u_{\min} \leq u_t \leq u_{\max}.$$

The bounds are selected from the nominal generated input as

$$u_{\max} = \max_t |u_t^{gen}| + 0.01, \quad u_{\min} = -u_{\max}.$$

The constrained optimization problem is implemented with CasADi `Opti` and solved using IPOPT. Only the first optimized control action is applied to the nonlinear system, and the optimization is repeated at the following time step.

5.2 MPC results and comparison with LQR

Figures 5.1 and 5.2 show the MPC tracking trajectories and error norms, respectively, for the same five perturbation scenarios considered in Task 3.

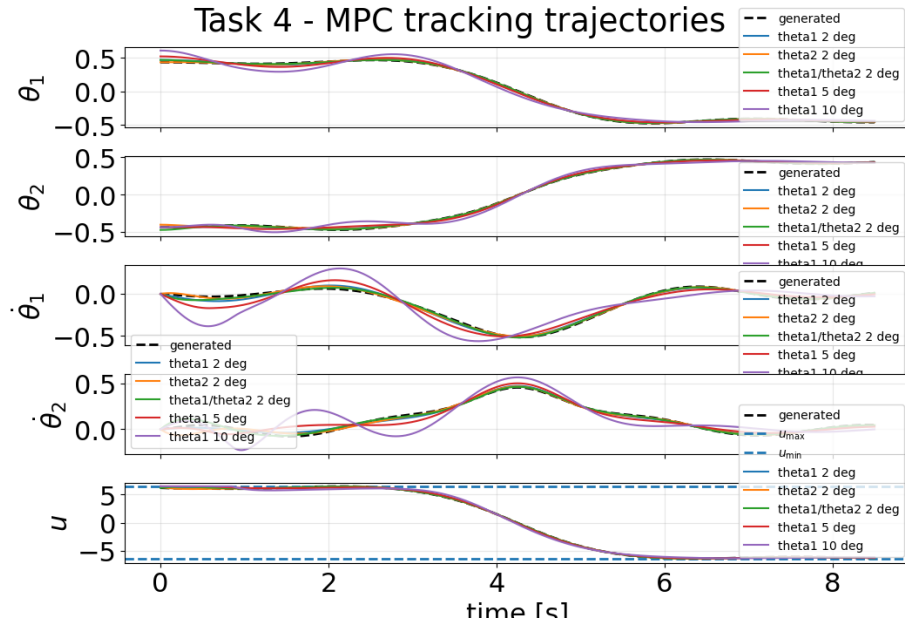


Figure 5.1: Task 4: generated trajectory and MPC tracking trajectories under five perturbations.

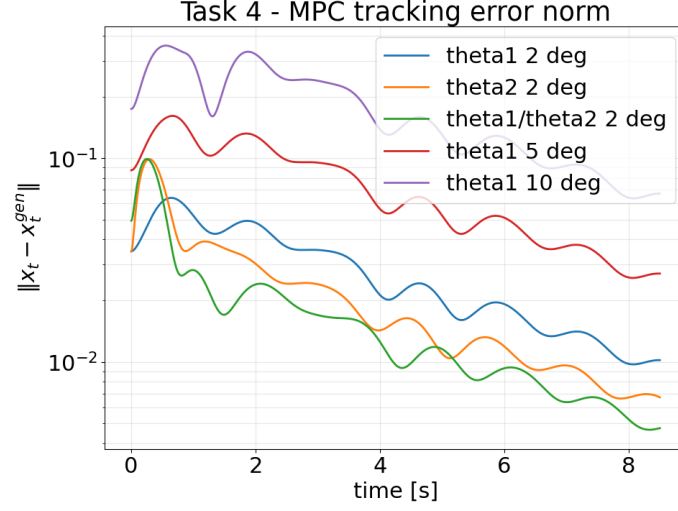


Figure 5.2: Task 4: MPC tracking error norm for five initial perturbations.

The MPC and LQR controllers use the same linearized model, state and input weights, and DARE-based terminal penalty, but they differ in two important aspects: the MPC uses a finite prediction horizon of $N_p = 5$ and explicitly enforces bounds on the actual control input. Therefore, the MPC is not expected to reproduce the LQR feedback action exactly. The comparison focuses on the tracking trajectories, the evolution of the tracking error, and the ability of the MPC to respect the imposed input constraints.

Chapter 6

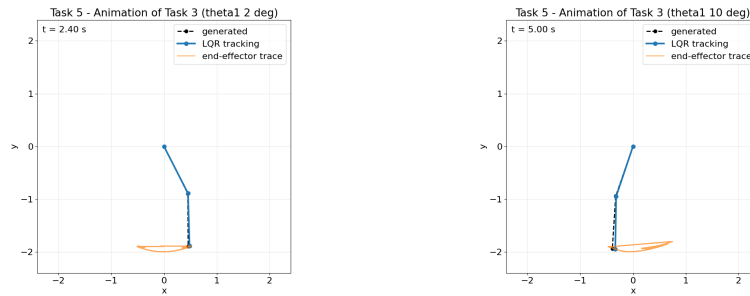
Task 5 – Animation

An animation of the two-link flexible arm was implemented to provide a qualitative visualization of the trajectory tracking results from Task 3. At each time step, the Cartesian coordinates of the two link endpoints are reconstructed from the angular states as

$$\begin{aligned}x_1 &= l_1 \sin(\theta_1), & y_1 &= -l_1 \cos(\theta_1), \\x_2 &= x_1 + l_2 \sin(\theta_1 + \theta_2), & y_2 &= y_1 - l_2 \cos(\theta_1 + \theta_2).\end{aligned}$$

The animation displays the nominal generated trajectory (dashed) and the LQR-controlled trajectory (solid), along with the trace of the end-effector position over time.

Two representative frames from the animation are shown in Figure 6.1. The left frame corresponds to the 2° perturbation on θ_1 at $t = 2.40$ s, when the arm is mid-transition: the controlled trajectory is almost superimposed on the generated one. The right frame shows the 10° perturbation case at $t = 5.00$ s, where a small but visible offset between the two arms is present due to the larger initial error, and the end-effector trace reflects a slightly wider path.



(a) θ_1 perturbation 2° , $t = 2.40$ s

(b) θ_1 perturbation 10° , $t = 5.00$ s

Figure 6.1: Task 5: animation frames of the LQR tracking simulation.

Conclusions

This project developed a complete optimal control framework for a flexible two-link robotic arm. The nonlinear Euler–Lagrange dynamics were discretized using Forward Euler with a sampling time of 10^{-2} s. CasADi was used to compute the Jacobians of the symbolic discrete-time dynamics via algorithmic differentiation, which were then used in the optimization, LQR, and MPC algorithms.

Two optimal trajectory generation problems were solved using a Newton-like method based on affine LQR subproblems, backward Riccati recursion, closed-loop rollout, and Armijo line search. In Task 1, a sigmoid-interpolated reference between two equilibria was used, with a simple reference-input rollout as the initial guess. In Task 2, a quasi-static trajectory was constructed as a dense sequence of equilibria. Since this curve is not necessarily dynamically feasible during the transition, an LTV-LQR controller was first used to simulate a feasible trajectory close to it. This trajectory was then used as the warm start for the Newton optimization.

The generated trajectory from Task 2 was then used as the nominal reference for tracking. An LTV-LQR controller and a constrained MPC controller were designed using the same linearized model and cost weights. The MPC was formulated in error coordinates with a prediction horizon of $N_p = 5$ and explicit bounds on the actual control input. Both controllers were tested under the same five perturbation scenarios ranging from 2° to 10° , allowing their tracking behaviour to be compared while accounting for the MPC input constraints.

The animation implemented in Task 5 confirmed the qualitative correctness of the tracking behavior, showing the arm following the generated trajectory closely even for the largest tested perturbation.

Overall, the results demonstrate the effectiveness of Newton-based optimal control for trajectory generation and of LTV-LQR and MPC for trajectory tracking in nonlinear robotic systems.

Bibliography

- [1] CasADi developers. CasADi – symbolic framework for automatic differentiation, 2024. Available at: <https://github.com/casadi/casadi>.
- [2] CasADi developers. CasADi documentation – Function objects and AD, 2024. Available at: <https://web.casadi.org/docs>.
- [3] Giuseppe Notarstefano. Optimal control and reinforcement learning – course slides, 2025. Master Degree in Automation Engineering, University of Bologna.